

# ROADMAP: C + DSAs (De Cero a Hardcore)

---

## PARTE 1: BASICO/INTERMEDIO (4-6 semanas)

### C - Fundamentos

1. Sintaxis basica: printf, scanf, tipos de datos (int, float, char, double)
2. Operadores: aritmeticos, logicos, relacionales, bitwise basicos
3. Control de flujo: if/else, switch, ternario
4. Loops: for, while, do-while, break, continue
5. Funciones: declaracion, definicion, paso por valor
6. Arrays: unidimensionales, bidimensionales, strings (char arrays)
7. Punteros basicos: declaracion, dereferencia, aritmetica basica
8. Structs basicos: declaracion, acceso a miembros

### DSAs - Fundamentos

1. Complejidad: Big O notation (tiempo y espacio)
2. Arrays y Strings:
  - Busqueda lineal
  - Two pointers
  - Sliding window basico
3. Sorting basico:
  - Bubble sort
  - Selection sort
  - Insertion sort
4. Busqueda:
  - Binary search (iterativo y recursivo)
5. Recursion basica: factorial, fibonacci, suma

### EXAMEN BASICO/INTERMEDIO

Implementa estos 5 problemas en C puro:

1. Problema de arrays: Rotar un array k posiciones
2. String manipulation: Verificar si una cadena es palindromo (case-insensitive)
3. Busqueda: Implementar binary search en un array ordenado
4. Recursion: Suma de digitos de un numero usando recursion

- 
5. Mini proyecto: Sistema de registro de estudiantes (usando arrays de structs, CRUD basico sin dinamico)
-

## PARTE 2: INTERMEDIO (6-8 semanas)

### C - Intermedio

1. Punteros avanzados: punteros a punteros, punteros a funciones
2. Memoria dinamica: malloc, calloc, realloc, free
3. Strings avanzado: manipulacion con punteros, string.h
4. Structs avanzados: structs anidados, arrays de structs, punteros a structs
5. Typedef y Enum: uso practico
6. Archivos: fopen, fread, fwrite, fclose
7. Preprocesador: #define, #ifdef, macros

### DSAs - Intermedio

1. Linked Lists:
  - Singly linked list (insertar, eliminar, buscar, reversa)
  - Doubly linked list
  - Circular linked list
2. Stacks: implementacion con arrays y linked lists
3. Queues: implementacion con arrays y linked lists, circular queue
4. Hashing basico:
  - Hash tables (implementacion simple)
  - Manejo de colisiones (chaining)
5. Sorting avanzado:
  - Merge sort
  - Quick sort
6. Recursion avanzada: backtracking basico

### EXAMEN INTERMEDIO

Implementa estos 6 problemas:

1. Linked List: Detectar ciclos (Floyd's algorithm)
2. Stack: Validar parentesis balanceados (multiples tipos: (), {}, [])
3. Queue: Implementar cola usando dos stacks
4. Hashing: Sistema de cache LRU (basico, tamano fijo)
5. Recursion: Generar todas las permutaciones de una cadena

- 
6. Mini proyecto: Editor de texto simple en consola (con undo/redondo usando stacks, guardar/cargar archivos)
-

## PARTE 3: AVANZADO (8-10 semanas)

### C - Avanzado

1. Bitwise operations: manipulacion avanzada de bits
2. Function pointers: callbacks, tablas de funciones
3. Memory management: memory leaks, valgrind
4. Optimization: inline, register, volatile
5. Unions: uso y diferencias con structs
6. Manejo de errores: errno, perror
7. Makefiles basicos

### DSAs - Avanzado

1. Trees:
  - Binary trees (traversals: inorder, preorder, postorder)
  - Binary Search Trees (BST): insertar, buscar, eliminar
  - AVL trees (self-balancing)
  - Heap (min-heap, max-heap, heap sort)
2. Graphs:
  - Representacion (adjacency matrix, adjacency list)
  - BFS (Breadth-First Search)
  - DFS (Depth-First Search)
  - Dijkstra's algorithm
  - Topological sort
3. Tries: implementacion y busqueda
4. Dynamic Programming basico:
  - Memoization
  - Tabulation
  - Problemas clasicos: knapsack, LCS, LIS
5. Greedy algorithms: conceptos y problemas clasicos

### EXAMEN AVANZADO

Implementa estos 7 problemas:

1. BST: Implementar BST completo con todas las operaciones + encontrar k-

esimo elemento mas pequeno

2. Heap: Implementar priority queue usando min-heap
  3. Graph - BFS/DFS: Encontrar caminos en un laberinto
  4. Graph - Dijkstra: Encontrar el camino mas corto en un grafo ponderado
  5. Trie: Sistema de autocompletado de palabras
  6. DP: Problema de la mochila (0/1 knapsack)
  7. Mini proyecto: Sistema de gestion de red social (grafo de usuarios, BFS para amigos en comun, Dijkstra para grados de separacion)
-

## | PARTE 4: HARDCORE (10-12 semanas)

### C - Hardcore

1. Memory alignment y padding
2. Atomics y concurrencia basica (si quieres pthreads)
3. Custom memory allocators
4. Bit manipulation avanzada: tecnicas de optimizacion
5. Profiling y debugging avanzado: gdb, valgrind avanzado
6. Sistema de modulos: headers guards, static/extern

### DSAs - Hardcore

1. Advanced Trees:
  - Red-Black trees
  - B-trees
  - Segment trees
  - Fenwick trees (Binary Indexed Tree)
2. Advanced Graphs:
  - Bellman-Ford
  - Floyd-Warshall
  - Kruskal's y Prim's (MST)
  - Strongly connected components (Kosaraju, Tarjan)
  - Network flow (Ford-Fulkerson)
3. Advanced DP:
  - DP con bitmasks
  - DP en arboles
  - Matrix chain multiplication
4. String algorithms:
  - KMP (pattern matching)
  - Rabin-Karp
  - Z-algorithm
5. Advanced Data Structures:
  - Skip lists
  - Bloom filters
  - Disjoint Set Union (Union-Find)

## EXAMEN Hardcore

Implementa estos 8 problemas + 1 proyecto final:

1. Red-Black Tree: Implementacion completa con todas las rotaciones
2. Segment Tree: Range sum queries con actualizaciones
3. Graph - MST: Implementar Kruskal con Union-Find
4. Graph - Network Flow: Maximo flujo en una red
5. String - KMP: Busqueda de patrones eficiente
6. DP avanzado: Edit distance (Levenshtein)
7. Custom allocator: Implementar tu propio malloc/free basico
8. Sistema completo: Implementar un cache LRU con todas las optimizaciones

## PROYECTO FINAL Hardcore

Sistema de base de datos en memoria con:

- B-tree para indexacion
  - Hash table para busquedas rapidas
  - Query parser basico
  - Transacciones con rollback
  - Persistencia en archivo
  - Memory pool custom
  - Todo en C puro, sin bibliotecas externas (excepto standard lib)
-

## | RECURSOS Y PRACTICA

### Para C:

- K&R "The C Programming Language" (libro clasico)
- Compiler: GCC con -Wall -Wextra -std=c11
- Debugger: GDB + Valgrind

### Para DSAs:

- LeetCode (filtrar por C)
- Codeforces (Division 2 y 3)
- CSES Problem Set
- CodeWars (como mencionaste)

### Sistema de evaluacion:

- Basico: Debes resolver sin ayuda en menos de 30 min cada problema
- Intermedio: Sin ayuda en menos de 45 min cada problema
- Avanzado: Sin ayuda en menos de 60 min cada problema
- Hardcore: Sin ayuda en menos de 90 min cada problema

Este roadmap te llevara de cero a ser un verdadero hardcore en C.

La clave esta en implementar todo desde cero (no usar bibliotecas externas) y entender cada byte de memoria que usas.